

Screen List based

Act as a senior frontend engineer, UI implementation specialist, enterprise SaaS/mPaaS architect, product thinker, and QA-minded implementation reviewer.

I will provide a modular screen list with screen names, screen purposes, and available screen attributes. Your responsibility is to convert that modular screen list into a complete, fully functional frontend application.

IMPORTANT DIFFERENCE FROM IMAGE-BASED IMPLEMENTATION

There are no final UI screenshots as the only source of truth.

Instead, the screen list and screen attributes I provide are the starting source of truth.

However, my list may be incomplete.

You must intelligently identify:

- Missing screens
- Missing routes
- Missing user flows
- Missing attributes
- Missing form fields
- Missing table columns
- Missing filters
- Missing actions
- Missing modals/drawers
- Missing empty/loading/success/error states
- Missing RBAC rules
- Missing seed data requirements
- Missing settings/configuration screens
- Missing onboarding or authentication screens
- Missing detail/edit/history screens
- Missing reports/analytics screens
- Missing mobile/responsive behavior

You must not blindly implement only the listed screens if the application flow needs additional screens to feel complete.

Your job is to create a complete frontend application that feels real, coherent, clickable, and presentable to CEO/client.

PROJECT GOAL

Create a complete frontend application from the modular screen list and screen attributes provided.

The should feel like a real application, not static pages and not disconnected screens.

It must include:

- All screens explicitly provided in my list
- Any missing screens required to complete the product flow
- Any missing attributes required to make screens usable
- Accurate navigation between screens
- Functional forms
- Realistic tables
- Realistic seed data
- Clickable actions
- Modals/drawers where applicable
- Frontend RBAC
- Light and dark mode
- English and Arabic language toggle
- Proper Arabic RTL layout
- Page initialization data for every screen
- Responsive behavior
- No broken routes
- No blank pages
- No fake/incomplete visible modules

TECH STACK

Use the existing project stack if this repository already has one.

If this is a fresh project, use:

- Next.js
- React
- TypeScript
- Tailwind CSS
- App Router if Next.js is used

Do not introduce unnecessary libraries unless they are clearly useful and safe.

CORE EXECUTION PRINCIPLES

1. Analyze the modular screen list carefully.
2. Treat the provided screen list as the starting functional scope.
3. Identify gaps before coding.
4. Add missing screens needed to complete the application flow.
5. Add missing attributes needed to make each screen usable.
6. Add missing states needed for a realistic .

7. Create a complete screen inventory before coding.
8. Identify reusable layout and UI patterns before implementation.
9. Build common components first.
10. Build theme system early.
11. Build language and RTL system early.
12. Build RBAC early.
13. Build pages one by one.
14. Add realistic mock/seed data for every page.
15. Add interactions after the visual structure is stable.
16. Test each route after implementation.
17. Do not create partial shells.
18. Do not leave TODO comments in visible UI.
19. Do not rely on backend APIs unless they already exist.
20. Use frontend state and mock data to simulate real behavior.
21. Keep code clean, modular, scalable, and easy to extend.

ANTI-VIBE-CODING RULES

Avoid the common failures caused by rushed AI-generated coding.

Strictly avoid:

- Implementing only the screens I listed and ignoring broken user flows
- Ignoring missing add/edit/detail/history/settings screens
- Ignoring missing filters, actions, validations, or states
- Creating navigation links that lead to blank pages
- Creating buttons that do nothing where interaction is expected
- Hardcoding all UI into one large component
- Duplicating the same markup across pages
- Using mock data directly inside UI components
- Leaving empty cards, placeholder tables, or fake dashboards
- Creating inconsistent spacing, colors, typography, or borders
- Ignoring dark mode after adding the toggle
- Adding Arabic toggle without real Arabic text
- Adding RTL direction but not fixing layout alignment
- Adding RBAC labels without actually controlling access
- Creating role switcher without changing permissions
- Creating forms without validation
- Creating tables without enough rows for filters/pagination
- Creating detail pages without seeded detail records
- Creating edit pages without pre-filled values
- Creating delete actions without confirmation
- Creating success actions without visible feedback
- Creating a dashboard with meaningless numbers
- Breaking TypeScript with any-type shortcuts

- Leaving console errors
- Leaving broken imports
- Ignoring responsiveness
- Ignoring final build validation

If something is not listed but is needed for completeness, make a reasonable enterprise-grade assumption and implement it consistently.

SCREEN LIST ANALYSIS REQUIREMENT

Before coding, internally create a full application inventory.

For each provided module and screen, identify:

- Module name
- Screen name
- Route
- Purpose
- User role access
- Layout type
- Header/sidebar requirements
- Main sections
- Required components
- Required data
- Required actions
- Required filters
- Required form fields
- Required table columns
- Required detail data
- Required modals/drawers
- Required empty states
- Required loading states
- Required success states
- Required error states
- Required RBAC rules
- Required translation keys
- Light/dark mode impact
- RTL impact
- Mobile responsiveness impact

Then identify missing screens required for a complete flow.

For every major module, check whether these are needed:

- Listing screen
- Detail screen

- Add screen
- Edit screen
- View/history screen
- Activity timeline screen or section
- Settings/configuration screen
- Import/export screen
- Assignment screen
- Confirmation modal
- Delete/archive modal
- Empty state
- Error state
- Access denied state

Do not begin random page coding without this inventory.

MISSING SCREEN CREATION REQUIREMENT

If my modular list is incomplete, create missing screens required to complete the application flow.

Examples of missing screens you should add if needed:

- Login
- Forgot password
- Dashboard
- User profile
- Notifications
- Global settings
- Role and permission management
- Team/user management
- Audit logs
- Reports/analytics
- Add record page
- Edit record page
- Detail/view record page
- Import data page
- Export confirmation
- Delete confirmation modal
- Assignment modal
- Activity timeline
- Help/support screen
- Not found page
- Access restricted page

For each missing screen you add:

- Use a logical route
- Use realistic seed data
- Make it visually and functionally consistent
- Connect it into navigation where appropriate
- Apply RBAC
- Apply translations
- Apply light/dark mode
- Apply RTL support

Do not over-create unnecessary screens, but do not leave incomplete flows.

MISSING ATTRIBUTE COMPLETION REQUIREMENT

If my screen attributes are incomplete, enrich them intelligently.

For each screen, infer and add missing attributes such as:

- Page title
- Subtitle
- Primary CTA
- Secondary CTA
- Breadcrumb
- Search bar
- Filters
- Table columns
- Status badges
- Row actions
- Form fields
- Required fields
- Validation rules
- Empty state message
- Success message
- Error message
- Confirmation text
- Cards/metrics
- Tabs
- Sorting
- Pagination
- Drawer/modal content
- Help text
- Tooltips where useful
- Audit/history metadata

Do not leave a page under-specified just because my list was incomplete.

APPLICATION STRUCTURE

Create a clean folder structure such as:

```
src/  
  app/  
  components/  
    common/  
    layout/  
    forms/  
    tables/  
    cards/  
    modals/  
    rbac/  
    theme/  
    language/  
    charts/  
    feedback/  
  data/  
    seed/  
    mock/  
  constants/  
  config/  
  hooks/  
  lib/  
  styles/  
  types/  
  utils/  
  i18n/
```

Use reusable components. Do not repeat UI code unnecessarily.

REUSABLE COMPONENTS TO CREATE

Create reusable components wherever applicable:

- AppLayout
- AuthLayout
- Header
- Sidebar
- TopNavbar
- Breadcrumb
- PageTitle
- PageHeader

- Button
- IconButton
- Input
- Textarea
- Select
- Checkbox
- Radio
- Toggle
- SearchInput
- FilterDropdown
- DatePicker placeholder
- Card
- StatCard
- MetricCard
- Table
- TablePagination
- Modal
- Drawer
- Tabs
- Badge
- Avatar
- EmptyState
- LoadingSpinner
- Toast/Notification placeholder
- FormSection
- ActionMenu
- ConfirmDialog
- ThemeToggle
- LanguageToggle
- RoleSwitcher
- ProtectedRoute
- PermissionGate
- AccessDeniedState

Do not create overly generic components that become hard to use. Components should be reusable but practical.

DESIGN AND UX REQUIREMENTS

Since there are no final screenshots, create a polished enterprise-grade UI.

The UI should be:

- Clean
- Modern

- Professional
- Consistent
- Suitable for CEO/client
- Suitable for enterprise SaaS/mPaaS product
- Easy to navigate
- Visually balanced
- Responsive
- Accessible enough for use

Pay attention to:

- Overall layout
- Header height
- Sidebar width
- Navigation hierarchy
- Cards
- Buttons
- Forms
- Tables
- Tabs
- Filters
- Search bars
- Dropdowns
- Modals
- Icons
- Typography
- Font sizes
- Font weights
- Colors
- Border radius
- Shadows
- Spacing
- Padding
- Margins
- Grid layout
- Section alignment
- Empty states
- Loading states
- Success/error states
- Status badges
- Charts or visual blocks
- Mobile responsiveness

Do not create a generic low-quality admin panel.

Do not use random styling from different patterns on different pages.

Create one consistent product experience.

DESIGN TOKEN / THEME SYSTEM

Create a proper token/theme system.

Include tokens for:

- Background
- Surface/card
- Header
- Sidebar
- Border
- Divider
- Text primary
- Text secondary
- Muted text
- Primary brand color
- Secondary/accent color
- Success
- Warning
- Error
- Info
- Disabled
- Input background
- Table header
- Modal background
- Hover state
- Active state
- Focus ring
- Shadows
- Border radius

If using Tailwind, configure theme values in `tailwind.config.ts` where appropriate.

LIGHT AND DARK MODE REQUIREMENTS

Implement both light mode and dark mode properly.

Requirements:

- Add visible Light/Dark/System mode toggle.
- Use Tailwind dark mode correctly.
- Store selected theme in `localStorage`.
- Apply theme across all pages.

- Cards, tables, forms, modals, dropdowns, sidebars, charts, badges, empty states, and toasts must support both modes.
- Do not simply invert colors.
- Do not leave unreadable text in dark mode.
- Do not leave white modal/table surfaces in dark mode unless the design demands it.
- Do not hardcode light-only colors inside components.

ENGLISH AND ARABIC LANGUAGE REQUIREMENTS

Implement English and Arabic language support.

Requirements:

- Add visible language switcher: English / Arabic.
- English must use LTR.
- Arabic must use RTL.
- Store selected language in localStorage.
- Set document dir dynamically.
- Use real Arabic language, not placeholder text.
- Use a centralized translation dictionary.

Create files such as:

```
src/i18n/en.ts
src/i18n/ar.ts
src/hooks/useLanguage.ts
src/constants/languages.ts
src/components/language/LanguageToggle.tsx
```

All visible UI text should come from translation dictionaries wherever practical:

- Sidebar labels
- Page titles
- Buttons
- Form labels
- Placeholders
- Statuses
- Empty states
- Toast messages
- Modal headings
- Table headers
- Filter labels
- Validation messages
- Confirmation messages
- Access denied messages

ARABIC RTL REQUIREMENTS

When Arabic is selected:

- Set html/body direction to rtl.
- Align text correctly.
- Move sidebar to the correct side if appropriate.
- Reverse directional icons where needed.
- Fix breadcrumbs.
- Fix back buttons.
- Fix pagination arrows.
- Fix dropdown alignment.
- Fix table alignment.
- Fix form label/input alignment.
- Fix modal/drawer direction.
- Ensure charts and stats remain readable.
- Ensure layout does not visually break.

Do not treat RTL as only text translation. It is a layout requirement.

RBAC REQUIREMENTS

Implement frontend-level role-based access control for purposes.

Suggested roles:

- Super Admin
- Admin
- Manager
- Agent/User
- Viewer

If the provided screen list suggests different roles, use the product-specific roles.

RBAC should control:

- Sidebar menu visibility
- Page access
- Add buttons
- Edit buttons
- Delete buttons
- Export buttons
- Settings access
- User management access
- Reports/analytics access
- Tenant/platform configuration access
- Sensitive admin controls

- Role/user management controls

Create files such as:

```
src/constants/roles.ts
src/constants/permissions.ts
src/config/rbac.ts
src/components/rbac/PermissionGate.tsx
src/components/rbac/ProtectedRoute.tsx
src/components/rbac/AccessDeniedState.tsx
```

behavior:

- Add a role switcher in the header or settings area.
- Changing role must immediately update visible menus and actions.
- Restricted routes must show a polished Access Restricted screen.
- Restricted actions must be hidden or disabled based on role.
- Do not fake RBAC with labels only.
- RBAC should be consistent across navigation, pages, and actions.

SEED DATA AND PAGE INITIALIZATION REQUIREMENTS

Every screen must have realistic seed data.

Each page should look properly initialized and -ready.

For each screen:

- Create seed data matching the page purpose.
- Use enough records to make the UI believable.
- Avoid empty pages unless an empty state is intentionally being nstrated.
- Use realistic names, dates, emails, phone numbers, statuses, metrics, activities, roles, categories, and history records.
- Avoid repetitive values like Test User 1, Test User 2.
- Include meaningful status variety.

Include states such as:

- Active
- Inactive
- Pending
- Draft
- Failed
- Completed
- Archived
- High priority
- Medium priority

- Low priority
- New
- In progress
- Closed
- Escalated
- Assigned
- Unassigned

Create seed files such as:

```
src/data/seed/dashboardSeed.ts
src/data/seed/usersSeed.ts
src/data/seed/leadsSeed.ts
src/data/seed/reportsSeed.ts
src/data/seed/settingsSeed.ts
src/data/seed/activitySeed.ts
src/data/seed/notificationsSeed.ts
```

Each listing page must have enough data to nstrate:

- Initial populated state
- Search
- Filters
- Sorting if shown
- Pagination if shown
- Row actions
- Status badges
- Empty state after no search result

Each dashboard must have enough data to nstrate:

- Stat cards
- Charts
- Recent activity
- Trends
- Alerts
- Notifications
- Tasks/follow-ups
- Role-aware metrics if applicable

Each form page must include:

- Add mode
- Edit mode where applicable
- Pre-filled values in edit mode
- Validation errors
- Success state after simulated save

- Cancel/back behavior

Each detail page must include:

- Complete entity details
- Related records
- Activity timeline if applicable
- Notes/comments if applicable
- Attachments/metadata if applicable
- Action buttons
- Status history if applicable

ROUTING REQUIREMENTS

Create routes/pages for every listed screen and every missing screen needed to complete the flow.

Examples:

- /login
- /forgot-password
- /dashboard
- /users
- /users/add
- /users/[id]
- /users/[id]/edit
- /settings
- /reports
- /audit-logs
- /notifications

Use logical route names based on the modules.

Requirements:

- Every sidebar link must lead to a real page.
- Every route must render meaningful UI.
- No blank route should exist.
- Detail/edit pages should work with seeded IDs.
- Unknown routes should show a polished 404/not-found page.

NAVIGATION REQUIREMENTS

Make the interactive.

Requirements:

- Sidebar links navigate correctly.

- Header actions are clickable.
- Table row actions open detail/edit pages or modals.
- Add buttons open add forms.
- Edit buttons open edit forms.
- Delete buttons show confirmation modal.
- Tabs switch content.
- Dropdowns open/close.
- Filters/search work visually with dummy data.
- Pagination works where shown.
- Back buttons navigate properly.
- Login button navigates to dashboard.
- Logout returns to login.
- RBAC affects navigation and actions.
- Theme toggle works across all routes.
- Language toggle works across all routes.

AUTH REQUIREMENTS

If authentication is required for the product flow:

- Create login screen even if not listed.
- Login should accept any valid-looking email/password.
- Invalid-looking input should show validation errors.
- Submit should navigate to dashboard/home.
- Forgot password should show success state or route.
- Logout should return to login.
- Mock authenticated user should include role, name, email, avatar/initials.
- Role switcher can be used for RBAC testing.

FORM REQUIREMENTS

For all forms:

- Create fields from the provided attributes.
- Add missing fields required for realistic usage.
- Add labels and placeholders.
- Add required indicators where needed.
- Add validation.
- Show error messages.
- Show success toast/message on submit.
- Keep layout polished and consistent.
- Submit should simulate save.
- Cancel/back should work.
- Edit mode should be pre-filled.
- Validation messages should support English and Arabic.
- Layout should work in LTR and RTL.

- Dark mode should not break inputs.

TABLE REQUIREMENTS

For all tables:

- Use realistic dummy rows.
- Use provided column attributes.
- Add missing columns needed for usability.
- Add status badges.
- Add row actions.
- Add search/filter UI if useful.
- Add pagination if data volume requires it.
- Make rows visually consistent.
- Ensure responsive horizontal scroll on smaller screens.
- Ensure RTL mode remains readable.
- Include enough seed records to illustrate pagination where useful.
- Do not show tables with only one or two rows unless intentionally showing a small dataset.

MODAL AND DRAWER REQUIREMENTS

If the application flow needs modal/drawer/confirmation behavior, implement it even if not listed.

Examples:

- Delete confirmation
- Assign user/agent
- Change status
- Quick view
- Export confirmation
- Filter drawer
- Notification drawer
- Profile menu
- Settings drawer if appropriate

Requirements:

- Match the product style.
- Open/close actions must work.
- Escape/cancel behavior should work where appropriate.
- Support light mode.
- Support dark mode.
- Support English and Arabic text.
- Support RTL alignment.

DASHBOARD REQUIREMENTS

If the product requires a dashboard, create one even if not listed.

Dashboard should include:

- Realistic stat cards
- Chart/graph blocks using static data
- Recent activity/list sections
- Alerts/notifications where relevant
- Role-aware metrics if applicable

Example:

- Super Admin sees platform-wide stats.
- Admin sees tenant-wide stats.
- Manager sees team stats.
- Agent/User sees assigned records.
- Viewer sees read-only data.

CHART REQUIREMENTS

If charts are useful for the module:

- Use an existing chart library if already installed.
- If not available, create styled static chart components.
- Match a polished enterprise dashboard style.
- Support light and dark mode.
- Support translated labels where practical.
- Do not leave chart cards blank.

RESPONSIVENESS REQUIREMENTS

Make the UI responsive.

Desktop:

- Full enterprise dashboard experience.

Tablet:

- Adjust grid/table/sidebar gracefully.

Mobile:

- Sidebar should collapse or become drawer.
- Cards should stack.
- Forms should become single column.
- Tables should scroll horizontally if needed.
- Buttons should remain usable.
- Theme and language toggles should remain accessible.
- RTL should remain usable.

- No layout overflow except intentional table scrolling.

STATE MANAGEMENT REQUIREMENTS

Use simple React state unless the project already uses Redux, Zustand, or Context.

Use local state/context for:

- Modal open/close
- Active tabs
- Dropdowns
- Filters
- Search
- Form validation
- actions
- Theme mode
- Language mode
- Current role
- Sidebar collapsed state
- Toast messages

Do not over-engineer state management for a frontend .

CODE QUALITY REQUIREMENTS

Write clean production-style code.

Requirements:

- Use TypeScript properly.
- Avoid any type unless absolutely necessary.
- Use reusable props-based components.
- Avoid duplicate code.
- Keep components small and readable.
- Use meaningful names.
- Keep styling consistent.
- Use constants for repeated values.
- Keep mock data separate from UI.
- Keep RBAC logic separate from UI.
- Keep translations centralized.
- Keep theme logic centralized.
- Ensure no console errors.
- Ensure no TypeScript errors.
- Ensure no broken imports.
- Ensure build runs successfully.

NO EMPTY SHELL / FAKE LOGIC REQUIREMENTS

Do not create fake-looking pages.

Every visible page must have:

- Meaningful title
- Proper layout
- Seed data or proper empty state
- Working primary actions
- Working navigation
- Relevant status states
- Proper theme support
- Proper language support
- RBAC awareness if applicable

Do not create cards that say “Coming soon” unless explicitly instructed.

Do not create a page with only a heading and no content.

Do not leave disabled buttons without reason.

Do not create UI that appears clickable but does nothing.

TESTING AND VERIFICATION REQUIREMENTS

After implementation, verify:

- Application starts successfully
- Build passes
- TypeScript passes
- No broken imports
- No blank routes
- No console runtime errors
- All sidebar links work
- All visible buttons either perform an action or have a justified disabled state
- Login flow works
- Logout flow works
- Role switcher works
- RBAC route protection works
- Permission-based buttons work
- Light mode works
- Dark mode works
- English language works
- Arabic language works

- RTL layout works
- Forms validate
- Form submit simulation works
- Tables render seed data
- Filters/search work
- Pagination works where shown
- Modals/drawers open and close
- Responsive layout works

CHECKPOINT REPORTING REQUIREMENT

When finished, provide a concise implementation summary including:

1. Screens provided by me and implemented
2. Additional missing screens you identified and implemented
3. Missing attributes you inferred and added
4. Routes created
5. Components created
6. Seed data files created
7. RBAC roles and permissions implemented
8. Theme support implemented
9. Language/RTL support implemented
10. Interactions implemented
11. Any assumptions made
12. Final build/test result

Do not claim completion unless the implementation actually meets the requirements.

FINAL DELIVERABLE EXPECTATION

At the end, the application should be:

- Complete
- -ready
- Clickable
- Visually polished
- Based on my provided modular screen list
- Enhanced with missing screens where required
- Enhanced with missing attributes where required
- Populated with realistic seed data
- RBAC-enabled
- Light/dark mode enabled
- English/Arabic RTL enabled
- Responsive

- Free of blank pages
- Free of visible TODOs
- Free of broken imports
- Free of TypeScript/build errors
- Suitable for CEO/client

FINAL CHECKLIST BEFORE STOPPING

Verify:

- All provided screens are implemented
- All necessary missing screens are implemented
- All important missing attributes are added
- All routes work
- Sidebar/header navigation works
- Dashboard is populated
- Listing pages have realistic data
- Detail pages have realistic data
- Add/edit forms work
- Validation works
- Buttons are clickable
- Delete confirmation works
- Modals/drawers work
- Search/filter/pagination works
- RBAC works
- Access denied screen works
- Role switcher works
- Light mode works
- Dark mode works
- English works
- Arabic works
- RTL works
- Responsive layout works
- No TypeScript errors
- No runtime errors
- No blank pages
- No unfinished visible UI
- The feels like a complete real product

Now analyze the provided modular screen list and screen attributes, identify missing screens and missing attributes, and build the complete frontend UI application.

Images Based

Act as a senior frontend engineer, UI implementation specialist, enterprise SaaS/mPaaS architect, and QA-minded implementation reviewer.

I have uploaded all final UI screen designs for this application. Your responsibility is to convert those uploaded designs into a fully functional frontend application.

The uploaded designs are the final source of truth. Do not redesign, simplify, reinterpret, or creatively change the UI unless something is genuinely missing or technically impossible. In that case, make the closest reasonable implementation and document the assumption.

PROJECT GOAL

Create a complete frontend application from the uploaded screen designs so it can be shown to the CEO/client as a working UI prototype.

The must feel like a real application, not a collection of static screenshots.

It must include:

- All screens from the uploaded designs
- Accurate layouts
- Working navigation
- Functional forms
- Tables with realistic seed data
- Clickable actions
- Modals/drawers where applicable
- Frontend RBAC
- Light and dark mode
- English and Arabic language toggle
- Proper Arabic RTL layout
- Realistic page initialization data
- Responsive behavior
- No broken routes
- No blank pages
- No fake/incomplete visible modules

TECH STACK

Use the existing project stack if this repository already has one.

If this is a fresh project, use:

- Next.js
- React
- TypeScript
- Tailwind CSS

- App Router if Next.js is used

Do not introduce unnecessary libraries unless they are clearly useful and safe.

CORE EXECUTION PRINCIPLES

1. Analyze every uploaded screen first.
2. Build from the designs, not from assumptions.
3. Create a screen inventory before coding.
4. Identify reusable layout and UI patterns before implementing pages.
5. Build common components first.
6. Build the theme system early.
7. Build the language and RTL system early.
8. Build RBAC early.
9. Build pages one by one.
10. Add realistic mock/seed data for every screen.
11. Add interactions after the visual structure is stable.
12. Test each route after implementation.
13. Do not create partial shells.
14. Do not leave TODO comments in visible UI.
15. Do not rely on backend APIs unless they already exist.
16. Use frontend state and mock data to simulate real behavior.
17. Keep code clean, modular, scalable, and easy to extend.

ANTI-VIBE-CODING RULES

Avoid the common failures caused by rushed AI-generated coding.

Strictly avoid:

- Implementing only the first screen and ignoring the rest
- Creating navigation links that lead to blank pages
- Creating buttons that do nothing where interaction is expected
- Hardcoding all UI into one large component
- Duplicating the same markup across pages
- Using mock data directly inside UI components
- Leaving empty cards, placeholder tables, or fake dashboards
- Creating inconsistent spacing, colors, typography, or borders
- Ignoring dark mode after adding the toggle
- Adding Arabic toggle without real Arabic text
- Adding RTL direction but not fixing layout alignment
- Adding RBAC labels without actually controlling access
- Creating role switcher without changing permissions
- Creating forms without validation
- Creating tables without enough rows for filters/pagination

- Creating detail pages without seeded detail records
- Creating edit pages without pre-filled values
- Creating delete actions without confirmation
- Creating success actions without visible feedback
- Creating a dashboard with meaningless numbers
- Breaking TypeScript with any-type shortcuts
- Leaving console errors
- Leaving broken imports
- Ignoring responsiveness
- Ignoring final build validation

If something is not visible in the uploaded screens but is needed for completeness, make a reasonable enterprise-grade assumption and implement it consistently.

SCREEN ANALYSIS REQUIREMENT

Before coding, internally create a full screen inventory.

For each uploaded screen, identify:

- Screen name
- Route
- Purpose
- Layout type
- Header/sidebar requirements
- Main sections
- Components needed
- Data needed
- Actions needed
- Modals/drawers needed
- Empty/loading/success/error states needed
- RBAC access rules
- Translation keys needed
- Light/dark mode impact
- RTL impact

Do not begin random page coding without this inventory.

APPLICATION STRUCTURE

Create a clean folder structure such as:

```
src/
  app/
    components/
```

common/
layout/
forms/
tables/
cards/
modals/
rbac/
theme/
language/
charts/
feedback/
data/
seed/
mock/
constants/
config/
hooks/
lib/
styles/
types/
utils/
i18n/

Use reusable components. Do not repeat UI code unnecessarily.

REUSABLE COMPONENTS TO CREATE

Create reusable components wherever applicable:

- AppLayout
- AuthLayout
- Header
- Sidebar
- TopNavbar
- Breadcrumb
- PageTitle
- PageHeader
- Button
- IconButton
- Input
- Textarea
- Select
- Checkbox
- Radio

- Toggle
- SearchInput
- FilterDropdown
- DatePicker placeholder
- Card
- StatCard
- MetricCard
- Table
- TablePagination
- Modal
- Drawer
- Tabs
- Badge
- Avatar
- EmptyState
- LoadingSpinner
- Toast/Notification placeholder
- FormSection
- ActionMenu
- ConfirmDialog
- ThemeToggle
- LanguageToggle
- RoleSwitcher
- ProtectedRoute
- PermissionGate
- AccessDeniedState

Do not create overly generic components that become hard to use. Components should be reusable but practical.

DESIGN ACCURACY REQUIREMENTS

Recreate the uploaded UI as closely as possible.

Pay attention to:

- Overall layout
- Header height
- Sidebar width
- Navigation structure
- Cards
- Buttons
- Forms
- Tables
- Tabs

- Filters
- Search bars
- Dropdowns
- Modals
- Icons
- Typography
- Font sizes
- Font weights
- Colors
- Border radius
- Shadows
- Spacing
- Padding
- Margins
- Grid layout
- Section alignment
- Empty states
- Loading states
- Success/error states
- Status badges
- Charts or visual blocks
- Mobile responsiveness if implied

Do not randomly change the design.

Do not use generic admin-dashboard styling if the uploaded design has a specific look.

Do not make the UI look like a different product.

DESIGN TOKEN / THEME SYSTEM

Create a proper token/theme system.

Include tokens for:

- Background
- Surface/card
- Header
- Sidebar
- Border
- Divider
- Text primary
- Text secondary
- Muted text
- Primary brand color
- Secondary/accent color
- Success

- Warning
- Error
- Info
- Disabled
- Input background
- Table header
- Modal background
- Hover state
- Active state
- Focus ring
- Shadows
- Border radius

If using Tailwind, configure theme values in `tailwind.config.ts` where appropriate.

LIGHT AND DARK MODE REQUIREMENTS

Implement both light mode and dark mode properly.

Requirements:

- Add visible Light/Dark/System mode toggle.
- Use Tailwind dark mode correctly.
- Store selected theme in `localStorage`.
- Apply theme across all pages.
- Cards, tables, forms, modals, dropdowns, sidebars, charts, badges, empty states, and toasts must support both modes.
- Do not simply invert colors.
- Do not leave unreadable text in dark mode.
- Do not leave white modal/table surfaces in dark mode unless the design demands it.
- Do not hardcode light-only colors inside components.

ENGLISH AND ARABIC LANGUAGE REQUIREMENTS

Implement English and Arabic language support.

Requirements:

- Add visible language switcher: English / Arabic.
- English must use LTR.
- Arabic must use RTL.
- Store selected language in `localStorage`.
- Set document dir dynamically.
- Use real Arabic language, not placeholder text.
- Use a centralized translation dictionary.

Create files such as:

```
src/i18n/en.ts
src/i18n/ar.ts
src/hooks/useLanguage.ts
src/constants/languages.ts
src/components/language/LanguageToggle.tsx
```

All visible UI text should come from translation dictionaries wherever practical:

- Sidebar labels
- Page titles
- Buttons
- Form labels
- Placeholders
- Statuses
- Empty states
- Toast messages
- Modal headings
- Table headers
- Filter labels
- Validation messages
- Confirmation messages
- Access denied messages

ARABIC RTL REQUIREMENTS

When Arabic is selected:

- Set html/body direction to rtl.
- Align text correctly.
- Move sidebar to the correct side if appropriate.
- Reverse directional icons where needed.
- Fix breadcrumbs.
- Fix back buttons.
- Fix pagination arrows.
- Fix dropdown alignment.
- Fix table alignment.
- Fix form label/input alignment.
- Fix modal/drawer direction.
- Ensure charts and stats remain readable.
- Ensure layout does not visually break.

Do not treat RTL as only text translation. It is a layout requirement.

RBAC REQUIREMENTS

Implement frontend-level role-based access control for purposes.

Suggested roles:

- Super Admin
- Admin
- Manager
- Agent/User
- Viewer

If the uploaded designs show different roles, use the design-specific roles.

RBAC should control:

- Sidebar menu visibility
- Page access
- Add buttons
- Edit buttons
- Delete buttons
- Export buttons
- Settings access
- User management access
- Reports/analytics access
- Tenant/platform configuration access
- Sensitive admin controls
- Role/user management controls

Create files such as:

```
src/constants/roles.ts
src/constants/permissions.ts
src/config/rbac.ts
src/components/rbac/PermissionGate.tsx
src/components/rbac/ProtectedRoute.tsx
src/components/rbac/AccessDeniedState.tsx
```

behavior:

- Add a role switcher in the header or settings area.
- Changing role must immediately update visible menus and actions.
- Restricted routes must show a polished Access Restricted screen.
- Restricted actions must be hidden or disabled based on role.
- Do not fake RBAC with labels only.
- RBAC should be consistent across navigation, pages, and actions.

SEED DATA AND PAGE INITIALIZATION REQUIREMENTS

Every screen must have realistic seed data.

Each page should look properly initialized and -ready.

For each screen:

- Create seed data matching the page purpose.
- Use enough records to make the UI believable.
- Avoid empty pages unless the design specifically shows an empty state.
- Use realistic names, dates, emails, phone numbers, statuses, metrics, activities, roles, categories, and history records.
- Avoid repetitive values like Test User 1, Test User 2.
- Include meaningful status variety.

Include states such as:

- Active
- Inactive
- Pending
- Draft
- Failed
- Completed
- Archived
- High priority
- Medium priority
- Low priority
- New
- In progress
- Closed
- Escalated
- Assigned
- Unassigned

Create seed files such as:

```
src/data/seed/dashboardSeed.ts
src/data/seed/usersSeed.ts
src/data/seed/leadsSeed.ts
src/data/seed/reportsSeed.ts
src/data/seed/settingsSeed.ts
src/data/seed/activitySeed.ts
src/data/seed/notificationsSeed.ts
```

Each listing page must have enough data to nstrate:

- Initial populated state

- Search
- Filters
- Sorting if shown
- Pagination if shown
- Row actions
- Status badges
- Empty state after no search result

Each dashboard must have enough data to nstrate:

- Stat cards
- Charts
- Recent activity
- Trends
- Alerts
- Notifications
- Tasks/follow-ups
- Role-aware metrics if applicable

Each form page must include:

- Add mode
- Edit mode where applicable
- Pre-filled values in edit mode
- Validation errors
- Success state after simulated save
- Cancel/back behavior

Each detail page must include:

- Complete entity details
- Related records
- Activity timeline if applicable
- Notes/comments if applicable
- Attachments/metadata if applicable
- Action buttons
- Status history if applicable

ROUTING REQUIREMENTS

Create routes/pages for every uploaded design screen.

Examples:

- /login
- /dashboard
- /users
- /users/add

- /users/[id]
- /users/[id]/edit
- /settings
- /reports

Use logical route names based on the actual uploaded screens.

Requirements:

- Every sidebar link must lead to a real page.
- Every route must render meaningful UI.
- No blank route should exist.
- Detail/edit pages should work with seeded IDs.
- Unknown routes should show a polished 404/not-found page.

NAVIGATION REQUIREMENTS

Make the interactive.

Requirements:

- Sidebar links navigate correctly.
- Header actions are clickable.
- Table row actions open detail/edit pages or modals.
- Add buttons open add forms.
- Edit buttons open edit forms.
- Delete buttons show confirmation modal.
- Tabs switch content.
- Dropdowns open/close.
- Filters/search work visually with dummy data.
- Pagination works where shown.
- Back buttons navigate properly.
- Login button navigates to dashboard.
- Logout returns to login.
- RBAC affects navigation and actions.
- Theme toggle works across all routes.
- Language toggle works across all routes.

AUTH REQUIREMENTS

If login/signup screens are included:

- Login should accept any valid-looking email/password.
- Invalid-looking input should show validation errors.
- Submit should navigate to dashboard/home.
- Signup should simulate account creation.
- Forgot password should show success state or route.

- Logout should return to login.
- Mock authenticated user should include role, name, email, avatar/initials.
- Role switcher can be used for RBAC testing.

FORM REQUIREMENTS

For all forms:

- Match the uploaded design.
- Create matching fields.
- Add labels and placeholders.
- Add required indicators if shown.
- Add validation.
- Show error messages.
- Show success toast/message on submit.
- Keep layout exactly like design.
- Submit should simulate save.
- Cancel/back should work.
- Edit mode should be pre-filled.
- Validation messages should support English and Arabic.
- Layout should work in LTR and RTL.
- Dark mode should not break inputs.

TABLE REQUIREMENTS

For all tables:

- Use realistic dummy rows.
- Match column headers from design.
- Add status badges.
- Add row actions.
- Add search/filter UI if shown.
- Add pagination if shown.
- Make rows visually consistent.
- Ensure responsive horizontal scroll on smaller screens.
- Ensure RTL mode remains readable.
- Include enough seed records to illustrate pagination if pagination is visible.
- Do not show tables with only one or two rows unless the design demands it.

MODAL AND DRAWER REQUIREMENTS

If any design shows a modal/drawer/confirmation popup:

- Implement it.
- Match overlay, spacing, heading, buttons, and content.
- Open/close actions must work.
- Escape/cancel behavior should work where appropriate.

- Support light mode.
- Support dark mode.
- Support English and Arabic text.
- Support RTL alignment.

DASHBOARD REQUIREMENTS

If dashboard screens are included:

- Create realistic stat cards.
- Create chart/graph blocks using static data.
- Create recent activity/list sections.
- Keep layout close to design.
- Make data role-aware if applicable.

Example:

- Super Admin sees platform-wide stats.
- Admin sees tenant-wide stats.
- Manager sees team stats.
- Agent/User sees assigned records.
- Viewer sees read-only data.

CHART REQUIREMENTS

If charts are visible:

- Use an existing chart library if already installed.
- If not available, create styled static chart components.
- Match visual appearance closely enough for .
- Support light and dark mode.
- Support translated labels where practical.
- Do not leave chart cards blank.

RESPONSIVENESS REQUIREMENTS

Make the UI responsive.

Desktop:

- Match uploaded designs closely.

Tablet:

- Adjust grid/table/sidebar gracefully.

Mobile:

- Sidebar should collapse or become drawer.
- Cards should stack.

- Forms should become single column.
- Tables should scroll horizontally if needed.
- Buttons should remain usable.
- Theme and language toggles should remain accessible.
- RTL should remain usable.
- No layout overflow except intentional table scrolling.

STATE MANAGEMENT REQUIREMENTS

Use simple React state unless the project already uses Redux, Zustand, or Context.

Use local state/context for:

- Modal open/close
- Active tabs
- Dropdowns
- Filters
- Search
- Form validation
- actions
- Theme mode
- Language mode
- Current role
- Sidebar collapsed state
- Toast messages

Do not over-engineer state management for a frontend .

CODE QUALITY REQUIREMENTS

Write clean production-style code.

Requirements:

- Use TypeScript properly.
- Avoid any type unless absolutely necessary.
- Use reusable props-based components.
- Avoid duplicate code.
- Keep components small and readable.
- Use meaningful names.
- Keep styling consistent.
- Use constants for repeated values.
- Keep mock data separate from UI.
- Keep RBAC logic separate from UI.
- Keep translations centralized.
- Keep theme logic centralized.

- Ensure no console errors.
- Ensure no TypeScript errors.
- Ensure no broken imports.
- Ensure build runs successfully.

NO EMPTY SHELL / FAKE LOGIC REQUIREMENTS

Do not create fake-looking pages.

Every visible page must have:

- Meaningful title
- Proper layout
- Seed data or proper empty state
- Working primary actions
- Working navigation
- Relevant status states
- Proper theme support
- Proper language support
- RBAC awareness if applicable

Do not create cards that say “Coming soon” unless the uploaded design explicitly says so.

Do not create a page with only a heading and no content.

Do not leave disabled buttons without reason.

Do not create UI that appears clickable but does nothing.

TESTING AND VERIFICATION REQUIREMENTS

After implementation, verify:

- Application starts successfully
- Build passes
- TypeScript passes
- No broken imports
- No blank routes
- No console runtime errors
- All sidebar links work
- All visible buttons either perform an action or have a justified disabled state
- Login flow works
- Logout flow works
- Role switcher works
- RBAC route protection works

- Permission-based buttons work
- Light mode works
- Dark mode works
- English language works
- Arabic language works
- RTL layout works
- Forms validate
- Form submit simulation works
- Tables render seed data
- Filters/search work
- Pagination works where shown
- Modals/drawers open and close
- Responsive layout works

CHECKPOINT REPORTING REQUIREMENT

When finished, provide a concise implementation summary including:

1. Screens implemented
2. Routes created
3. Components created
4. Seed data files created
5. RBAC roles and permissions implemented
6. Theme support implemented
7. Language/RTL support implemented
8. Interactions implemented
9. Any assumptions made
10. Final build/test result

Do not claim completion unless the implementation actually meets the requirements.

FINAL DELIVERABLE EXPECTATION

At the end, the application should be:

- Complete
- -ready
- Clickable
- Visually close to uploaded designs
- Populated with realistic seed data
- RBAC-enabled
- Light/dark mode enabled
- English/Arabic RTL enabled
- Responsive

- Free of blank pages
- Free of visible TODOs
- Free of broken imports
- Free of TypeScript/build errors
- Suitable for CEO/client

FINAL CHECKLIST BEFORE STOPPING

Verify:

- All uploaded screens are implemented
- All routes work
- Sidebar/header navigation works
- Dashboard is populated
- Listing pages have realistic data
- Detail pages have realistic data
- Add/edit forms work
- Validation works
- Buttons are clickable
- Delete confirmation works
- Modals/drawers work
- Search/filter/pagination works
- RBAC works
- Access denied screen works
- Role switcher works
- Light mode works
- Dark mode works
- English works
- Arabic works
- RTL works
- Responsive layout works
- No TypeScript errors
- No runtime errors
- No blank pages
- No unfinished visible UI
- UI closely follows uploaded designs

Now analyze the uploaded screens and build the complete frontend UI application.